in golang with code generation, can you get close to .NET/C# Vogen? Especially, how to support adding the Validate method when Golang doesn't have partial methods? Maybe the code generator should emit a parse method accepting an func to mimic partial behavior.

Yes, you can get remarkably close to the developer experience and safety of C#'s `Vogen` (Value Object Generator) in Go, even without partial classes or partial methods.

In Go, the idiomatic way to handle this is by separating the **publicly exposed, safe constructor** from the **underlying type definitions**, and using code generation to bridge the gap.

Here is how you can mimic that `Vogen` behavior—including the custom validation—using Go's type system and code generation patterns.

The Core Strategy: The "Wrapper + Extender" Pattern

Since Go lacks `partial`, we cannot let the user write half a struct and the generator write the other half in the same package easily without conflicts. Instead, the generator creates a **factory function (Constructor)** that explicitly forces the user's custom validation logic to run.

Instead of partial methods, we use a **Validation Hook** approach. The generator looks for a specific validation function signature in your handwritten code. If it exists, the generated constructor calls it.

How It Looks in Practice

1. The Handwritten Code (`user_id.go`)

You define your domain type and the validation function. By convention, you tag the type so your generator knows to process it.

Go

```go
package domain

//go:generate my-vogen -type=UserID

// UserID is our Value Object wrapper
type UserID struct{ value string }

// Value returns the underlying primitive (mimicking Vogen's .Value)
func (u UserID) Value() string {
        return u.value
}

// Validate is our "partial method" equivalent.
// The generator will detect this function and inject it into the Parse/New
func validateUserID(v string) error {
        if len(v) == 0 {
                return errors.New("UserID cannot be empty")
        }
        return nil
}
```

2. The Generated Code (`user_id_vogen.go`)

The code generator reads your file, sees `validateUserID` , and spits out the safe constructor.

Go

```go
// Code generated by my-vogen. DO NOT EDIT.
package domain

import "errors"

// NewUserID is the safe factory method (equivalent to Vogen's From/Parse)
func NewUserID(v string) (UserID, error) {
        // 1. Run the handwritten validation hook
        if err := validateUserID(v); err != nil {
                return UserID{}, err
        }

        // 2. Return the valid Value Object
        return UserID{value: v}, nil
}
```

Alternative: The Functional Parse Method (Your Idea)

Your idea of passing a `func` into a generic parse method is also highly viable, especially if you want to avoid AST parsing (Abstract Syntax Tree) in your generator to look for `validateUserID`.

If you want a highly reusable, generic approach, you can generate a **Parse Helper** like this:

Go

```go
// Generated Code
func ParseUserID(v string, validationHook func(string) error) (UserID, erro
        if err := validationHook(v); err != nil {
                return UserID{}, err
        }
        return UserID{value: v}, nil
}
```

Why the first approach is usually preferred in Go:
While the functional approach works, it forces the consumer of the `UserID` struct to pass the validation function *every time* they want to create an instance, which breaks encapsulation. Keeping the validation bound inside the domain package (via the generated `NewUserID`) keeps the API clean for the rest of your application.

Existing Go Tools That Do This

If you don't want to write the AST parsing generator from scratch, the Go ecosystem already has tools that mimic C# Value Objects perfectly:

1. `danwt/munch` **or custom** `go generate` **scripts:** Most Go developers use the `text/template` and `go/ast` standard libraries to build a small internal CLI that searches for types with a specific comment (e.g., `//vogen:string`) and generates the constructors.
2. `loov/hrd` **or** `derision-test/go-mockery` **style tooling:** You can use structural tags to generate the types.

Summary of Differences

Summary of Differences

Feature	C# Vogen	Go Code Gen
Enforcement	`readonly struct` prevents parameterless `new()`	Unexported struct fields prevent manual initialization outside the package.
Validation Hook	`partial void Validate()`	Generator detects `validate[Type]()` via AST parsing.
Compilation	Source Generators (Roslyn) run in real-time.	`go generate` must be run manually or via IDE file watchers.

By keeping the inner field of your struct unexported (e.g., `type UserID struct{ value string }`), no one outside your domain package can bypass your generated `NewUserID` constructor, giving you the exact same type-safety guarantees as .NET's Vogen.

in alternative 1, why not have the Value be code generated as well?

That is actually a great observation. In a production-ready Go value object generator, **the `Value()` method (and the struct definition itself) absolutely *should* be code-generated.** The reason it was split in the first example was simply to show the conceptual separation of handwritten logic versus generated logic, but in practice, writing the struct manually defeats half the purpose of automation.

Here is how a highly optimized Vogen-like tool handles this in Go to maximize automation while keeping validation flexible.

The Optimized "Single-Line Declaration" Pattern

Instead of writing the struct and the getter method by hand, you only write a single line of code (a comment or a dummy type) and the handwritten validation function. The generator takes care of everything else.

1. The Handwritten Code (`user_id.go`)

You just declare the intent and the validation function. You don't even define the struct fields or the `Value()` receiver.

Go

```go
package domain

//go:generate my-vogen -name=UserID -underlying=string

// validateUserID is detected by the generator based on the name mapping.
func validateUserID(v string) error {
        if len(v) == 0 {
                return errors.New("UserID cannot be empty")
        }
        return nil
}
```

2. The Entirely Generated Code (`user_id_vogen.go`)

The generator now outputs the struct, the constructor, *and* the getter.

Go

```go
// Code generated by my-vogen. DO NOT EDIT.
package domain

import "errors"

// UserID is completely generated
type UserID struct {
        value string
}

// Value is completely generated
func (u UserID) Value() string {
```

```
func (u UserID) Value() string {
        return u.value
}

// NewUserID wraps the custom validation
func NewUserID(v string) (UserID, error) {
        if err := validateUserID(v); err != nil {
                return UserID{}, err
        }
        return UserID{value: v}, nil
}
```

Why This is Much Better

1. **Zero Boilerplate:** You don't have to write `func (u UserID) Value() string { return u.value }` for 50 different domain IDs.
2. **Standardization:** Every single value object in your codebase will look, feel, and behave exactly the same way.
3. **Encapsulation by Default:** Because the generator controls the struct definition, it ensures the internal `value` field is always lowercase (unexported). This guarantees that code outside the `domain` package *cannot* bypass validation by doing `u := UserID{value: ""}`. They are forced to use `NewUserID()`.

The One Catch: Circular Detection

The only reason some Go generators ask you to write the `type UserID struct{ value string }` by hand is to make IDE integration smoother *before* the generator runs.

If the struct doesn't exist yet, your handwritten `validateUserID` function will show a compiler error in your IDE saying `"UserID is undefined"` until you run `go generate`. By generating the whole thing, you just have to get used to running `go generate ./...` frequently, which aligns perfectly with how C# source generators update as you type!